

阿里

如何解决消息堆积的问题？

🔥 薪资：35K

★ 岗位：高级开发

消息堆积概念

消费者在消费的过程中，**消费的速度跟不上服务端的发送速度**，未处理的消息会越来越多，消息出现堆积进而会造成消息消费延迟。

RocketMQ、Kafka 具备堆积的能力，但是以下场景需要重点关注消息堆积和延迟的问题：

1. 业务系统上下游能力不匹配造成的持续堆积，且无法自行恢复。
2. 业务系统对消息的消费实时性要求较高，即使是短暂的堆积造成的消息延迟也无法接受。



消息堆积的主要瓶颈

客户端使用 Push 模式 启动后，消费消息时，分为以下两个阶段：

阶段一：拉取消息

客户端通过长轮询批量拉取的方式从 Broker 服务端获取消息，将拉取到的消息缓存到本地缓冲队列中。客户端批量拉取消息，常见内网环境下都会有很高的吞吐量，例如：1个单线程单分区的低规格机器（4C8GB）可以达到几万 TPS，如果是多个分区可以达到几十万 TPS。所以这一阶段一般不会成为消息堆积的瓶颈。

长轮询：Consumer主动发起请求到Broker，正常情况下Broker响应消息给Consumer；在没有消息或者其他一些特殊场景下，可以将请求阻塞在服务端延迟返回。长轮询不是一种Push模式，而是Pull的一个变种



消息堆积的主要瓶颈

阶段二：消费消息

提交消费线程，客户端将本地缓存的消息提交到消费线程中，使用业务消费逻辑进行处理。此时客户端的消费能力就完全依赖于业务逻辑的复杂度（**消费耗时**）和消费逻辑**并发度**了。如果业务处理逻辑复杂，处理单条消息耗时都较长，则整体的消息吞吐量肯定不会高，此时就会导致客户端本地缓冲队列达到上限，停止从服务端拉取消息。

通过以上客户端消费原理可以看出，消息堆积的主要瓶颈在于本地客户端的消费能力，即消费耗时和消费并发度。

想要避免和解决消息堆积问题，必须合理的控制消费耗时和消息并发度，其中消费耗时的优先级高于消费并发度，必须先保证**消费耗时**的合理性，再考虑**消费并发度**问题。



消费耗时导致的消息堆积

影响消费耗时的消费逻辑主要分为 CPU 内存计算和外部 I/O 操作，通常情况下代码中如果没有复杂的递归和循环的话，内部计算耗时相对外部 I/O 操作来说几乎可以忽略。

外部 I/O 操作通常包括如下业务逻辑：

- 读写外部数据库，例如 MySQL 数据库读写。
- 读写外部缓存等系统，例如 Redis 读写。
- 下游系统调用，例如 Dubbo 调用或者下游 HTTP 接口调用。

这类外部调用的逻辑和系统容量需要提前梳理，掌握每个调用操作预期的耗时，这样才能判断消费逻辑中 I/O 操作的耗时是否合理。

通常消费堆积都是由于这些下游系统出现了服务异常、容量限制导致的消费耗时增加。

例如：某业务消费逻辑中需要调用下游 Dubbo 接口，单次消费耗时为 20 ms，平时消息量小未出现异常。业务侧进行大促活动时，下游 Dubbo 服务未进行优化，消费单条消息的耗时增加到 200 ms，业务侧可以明显感受到消费速度大幅下跌。此时，通过提升消费并行度并不能解决问题，需要大幅提高下游 Dubbo 服务性能才行。

消费并发度导致的消息堆积

绝大部分消息消费行为都属于 IO 密集型，即可能是操作数据库，或者调用 RPC，这类消费行为的消费速度在于后端数据库或者外系统的吞吐量，通过增加消费并行度，可以提高总的消费吞吐量，但是并行度增加到一定程度，反而会下降。

所以，应用必须要设置合理的并行度。如下有几种修改消费并行度的方法：

- 同一个 ConsumerGroup 下，通过增加 Consumer 实例数量来提高并行度（需要注意的是超过订阅队列数的 Consumer 实例无效）。可以通过加机器，或者在已有机器启动多个进程的方式。
- 提高单个 Consumer 实例的消费并行线程，通过修改参数 `consumeThreadMin`、`consumeThreadMax` 实现。

如何解决消息堆积问题

一、确认消息的消费耗时是否合理

首先，我们需要查看消费耗时，确认消息的消费耗时是否合理。查看消费耗时一般来讲有两种方式：

1、打印日志

```
public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt> msgs,
ConsumeConcurrentlyContext context) {
    try {
        for (MessageExt messageExt : msgs) {
            long start = System.currentTimeMillis();
            // TODO 业务逻辑
            logger.info("MessageId:" + messageExt.getMsgId() + " costTime:" +
(System.currentTimeMillis() - start));
        }
        return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
    } catch (Exception e) {
        logger.error("consumeMessage error:", e);
        return ConsumeConcurrentlyStatus.RECONSUME_LATER;
    }
}
```

如何解决消息堆积问题

2、查看消息轨迹

当确定好消费耗时后，可以根据耗时大小，采取不同的措施。

若查看到消费耗时较长，则需要查看客户端 JVM 堆栈信息排查具体业务逻辑，并优化消费逻辑。若查看到消费耗时正常，则有可能是因为消费并发度不够导致消息堆积，需要逐步调大消费线程或扩容节点来解决。

RocketMQ控制台 运维 驾驶舱 集群 主题 消费者 生产者 消息 消息轨迹 更换语言

MESSAGE KEY MESSAGE ID

Only Return 64 M

Topic: TopicT

7F00

Message ID	Tag	Message Key	StoreTime	StoreHost	costTime	status	traceType
7F000001EC2018B4AAC22220E1740000	TagA	a8f30e01-3a7d-486a-a10e-41a0b30f3256	2023-10-07 15:03:00	192.168.31.228:10911	29ms	Sucess	Pub
7F000001EC2018B4AAC22220E1740000		a8f30e01-3a7d-486a-a10e-41a0b30f3256	2023-10-07 15:03:30	192.168.31.228	0ms	Sucess	SubBefore
7F000001EC2018B4AAC22220E1740000		a8f30e01-3a7d-486a-a10e-41a0b30f3256	2023-10-07 15:03:43	192.168.31.228	513ms	Sucess	SubAfter

关闭

如何解决消息堆积问题

二、查看客户端 JVM 的堆栈

假如消费耗时非常高，需要查看 Consumer 实例 JVM 的堆栈。

通过 `jps -m` 或者 `ps -ef | grep java` 命令获取当前正在运行的 Java 程序，通过启动主类即可获得应用的进程 pid；

通过 `jstack pid > stack.log` 命令获取线程的堆栈。

执行以下命令，查看 `ConsumeMessageThread` 的信息。

```
cat stack.log | grep ConsumeMessageThread -A 10 --color
```

通过JVM堆栈分析消费耗时

常见的异常堆栈信息如下：

示例1：空闲无堆积的堆栈。

消费空闲情况下消费线程都会处于 WAITING 状态等待从消费任务队里中获取消息。

```
"ConsumeMessageThread_myconsumerGroup_2" #40 prio=5 os_prio=31 tid=0x000007fcb14939000 nid=0x14f03
waiting on condition [0x00000000310261000]
  java.lang.Thread.State: WAITING (parking)
    at sun.misc.Unsafe.park(Native Method)
      - parking to wait for <0x0000000076eb84328> (a
java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
    at java.util.concurrent.locks.LockSupport.park(LockSupport.java:175)
    at
java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject.await(AbstractQueuedSynchronizer.
java:2039)
    at java.util.concurrent.LinkedBlockingQueue.take(LinkedBlockingQueue.java:442)
    at java.util.concurrent.ThreadPoolExecutor.getTask(ThreadPoolExecutor.java:1074)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1134)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:750)
--
```

通过JVM堆栈分析消费耗时

示例2：消费逻辑有抢锁休眠等待等情况。
消费线程阻塞在内部的一个睡眠等待上，导致消费缓慢。

```
"ConsumeMessageThread_myconsumerGroup_2" #40 prio=5 os_prio=31 tid=0x00007f8ef110c800 nid=0x14a03
waiting on condition [0x00000000310405000]
  java.lang.Thread.State: TIMED_WAITING (sleeping)
    at java.lang.Thread.sleep(Native Method)
    at org.apache.rocketmq.example.quickstart.Consumer.lambda$main$0(Consumer.java:38)
    at org.apache.rocketmq.example.quickstart.Consumer$$Lambda$1/123961122.consumeMessage(Unknown
Source)
    at
org.apache.rocketmq.client.impl.consumer.ConsumeMessageConcurrentlyService$ConsumeRequest.run(ConsumeMe
ssageConcurrentlyService.java:402)
    at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:511)
    at java.util.concurrent.FutureTask.run(FutureTask.java:266)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:750)
```

通过JVM堆栈分析消费耗时

示例3：消费逻辑操作数据库等外部存储卡住。

消费线程阻塞在外部的 HTTP 调用上，导致消费缓慢。

```
"ConsumeMessageThread_myconsumerGroup_1" #36 prio=5 os_prio=31 tid=0x00007f7d3e9e8000 nid=0x1520b
runnable [0x000000030f040000]
  java.lang.Thread.State: RUNNABLE
    at java.net.SocketInputStream.socketRead0(Native Method)
    at java.net.SocketInputStream.socketRead(SocketInputStream.java:116)
    at java.net.SocketInputStream.read(SocketInputStream.java:171)
    at java.net.SocketInputStream.read(SocketInputStream.java:141)
    at java.io.BufferedInputStream.fill(BufferedInputStream.java:246)
    at java.io.BufferedInputStream.read1(BufferedInputStream.java:286)
    at java.io.BufferedInputStream.read(BufferedInputStream.java:345)
    - locked <0x0000000076b9470a0> (a java.io.BufferedInputStream)
    at sun.net.www.http.HttpClient.parseHTTPHeader(HttpClient.java:779)
```

总结

1. 客户端使用 Push模式 启动后，消费消息时，分为以下两个阶段：拉取消息和消费消息。
2. 客户端消费原理可以看出，消息堆积的主要瓶颈在于本地客户端的消费能力，即消费耗时和消费并发度。
3. 首先分析消费耗时，然后根据耗时大小，采取不同的措施。
 - 若查看到消费耗时较长，则查看客户端堆栈信息排查具体业务逻辑，并优化消费逻辑。
 - 若查看到消费耗时正常，则有可能是因为消费并发度不够导致消息堆积，需要逐步调大消费线程或扩容节点来解决。